How All the Formulas Work

Universe 3 — Reinsurance Treaty Pricing & Modelling Platform

Step-by-step reference for every numeric calculation in the codebase.
Covers shared math primitives, proportional treaty engine, non-proportional layer pricing (burning cost, Pareto, exposure rating), chain-ladder development, dashboard aggregates, FX conversion, and pricing verification.

Generated 2026-05-13.

Contents

Each formula entry follows the same structure: *where it lives*, *what it computes*, *inputs*, *the formula itself*, *edge cases*, and *who consumes the result*. File and line references appear in monospace blue.

1. Shared core math primitives

These five primitives live in `shared/pricingMath.js`. They are the canonical reference: the client uses them for live calculation, and the server's `pricingVerifier.js` imports the same functions to spot-check that stored NP pricing outputs match what the formulas would produce. Drift between the two surfaces is what the `X-Pricing-Drift-Count` header and the optional `PRICING_STRICT` gate (section 7) measure.

1.1 layerHit — non-proportional loss allocation

`shared/pricingMath.js:29–33`

The amount of a single ground-up loss that falls inside a layer defined by a deductible `D` (attachment) and limit `L` (layer width).

```
layerHit = max(0, min(loss − D, L))
```

`loss` currency — ground-up loss (usually inflation-adjusted from claims history)

`D` currency — layer attachment point

`L` currency — layer width (NOT top of layer)

Edge cases. Non-finite inputs return 0. A `limit ≤ 0` returns 0 (invalid layer). Negative deductibles are tolerated (rare but legal — represents a layer that picks up from the first dollar).

Consumed by. Burning-cost engine (cuts every selected historical loss to the layer), Pareto pricing (layer LEV), cat exposure rating (return-period curve integration), server-side verification.

1.2 weightedAverage

`shared/pricingMath.js:44–57`

Weighted mean of two parallel arrays. Returns `null` when every weight is ≤ 0, which lets callers distinguish "no data" from "the data averaged to zero".

```
weightedAverage = Σ(values[i] · weights[i]) / Σ(weights[i])
```

Edge cases. Zero or negative weights and non-finite values are skipped. Total weight ≤ 0, or mismatched array lengths, returns `null`.

Consumed by. Weighted age-to-age LDFs in chain ladder (§4.2). Risk-premium blending also uses it in places, though most blending now flows through the three-way `deriveComponentTotal` instead.

1.3 annualiseLoss

`shared/pricingMath.js:84–87`

Spread an observed total loss across an observation window to get an annual expected loss.

```
annualLoss = totalLoss / years
```

Calibration trap

`years` is the **total calendar window**, not the count of years with non-zero losses. Using only active years overstates expected loss by a factor of `(totalYears / activeYears)`. The burning-cost engine in §3.1 always passes the full observation window for exactly this reason.

Edge cases. `years ≤ 0` returns 0; non-finite inputs return 0.

1.4 ROL ↔ premium conversions

`shared/pricingMath.js:98–114`

Two inverse helpers for layer pricing. ROL ("rate-on-line") is the dimensionless ratio *premium / limit*.

```
ROL     = annualLoss / limit
premium = ROL × limit
```

Edge cases. `limit ≤ 0` returns 0 in both directions.

Consumed by. Burning-cost layer pricing, dashboard aggregates (when converting layer premium to ROL for display), `fqHelpers.fqLayerToXY` for the benchmark curve.

1.5 clamp

`shared/pricingMath.js:124–127`

```
clamp(n, min, max) = min(max(n, min), max)
```

NaN passes through unchanged — the function does not silently coerce. Used for clamping user-entered weights to [0, 100] and loadings to [0, 99].

1.6 applyLoading — target loss-ratio multiplier

`shared/pricingMath.js:148–157`

Convert a pure-loss rate to a gross rate by dividing out the target loss ratio.

```
grossRate = pureRate / (1 − clamp(loadingPct, 0, 99) / 100)
```

`pureRate` decimal, e.g. `0.03` for a 3% pure ROL
`loadingPct` percentage points, e.g. `20` for "load by 20"

Edge cases. Non-finite `pureRate` returns 0. Negative loading is clamped to 0 (which is the identity). `loadingPct ≥ 100` **throws a** `RangeError` — this is a deliberate hardening so that bad inputs surface upstream rather than being silently treated as zero.

1.7 deriveComponentTotal — three-way blend with loading

`shared/pricingMath.js:182–200`

The canonical NP final-pricing formula *per layer-component* (Risk or Cat). It blends three actuarial methods using user-supplied weights, then applies an expense loading.

```
1. sumW = wBurn + wPareto + wExposure
2. if sumW ≤ 0 → return 0  // no method selected
3. blended = (wBurn·burn + wPareto·pareto + wExposure·exposure) / sumW
4. grossRate = applyLoading(blended, loading)
5. return grossRate
```

`burn, pareto, exposure` decimals — the three component rates (ROL or rate on EGNPI)
`wBurn, wPareto, wExposure` percentage points 0–100; need not sum to 100 — the function normalises by their sum
`loading` percentage points 0–99

Input parsing. Inputs may arrive as formatted strings ("10.50%", "1,234"). The function strips percent signs and thousands separators before `parseFloat`.

Edge cases. All weights zero returns 0. A very small but non-zero `sumW` can produce a large `grossRate` — this is mathematically correct but rarely intentional, so callers should treat tiny weight sums as a UI warning.

Consumed by. `NpFinalPricing.jsx` (client computes per-layer risk and cat totals), the server's `pricingVerifier.js` (re-derives stored `total_price` from components + weights — see §7), and `fqHelpers.fqQuoteLayerDerived` for benchmark pricing curves.

2. Proportional treaty engine

The proportional engine lives in `client/src/logic/propTreatyEngine.js`. It runs on the client, applies sequentially (cap the loss → compute commission → compute profit commission → compute loss participation), and is fed by `buildTreatyTerms()` which flattens whatever shape the contract record happens to use into a single `t` object.

2.1 EPI — 100% earned premium for a proportional treaty

`server/src/routes/home.js:162 · server/src/routes/dashboard.js:98`

> EPI_{100} = quota_share_epi + surplus_epi

Stored on `contract_prop_details`. If the detail row is null or both columns are zero, dashboard aggregates fall back to the legacy `contract_pricing_outputs.epi`.

2.2 Loss cap — applied before commission/PC/LP

`client/src/logic/propTreatyEngine.js:15–24`

Cap claims at either an explicit percentage of premium, or an agreed annual loss (AAL) converted to a percentage. The capped value flows into every downstream calculation.

```
capPct =
   t.loss_cap_pct                                  if set
   (t.aal / prem) × 100                            else if aal > 0
   0                                                otherwise

cappedClaims = (capPct > 0) ? min(claims, prem × capPct / 100) : claims
```

Edge cases. `prem ≤ 0` short-circuits and returns the original claims.

2.3 Commission — fixed or sliding

`client/src/logic/propTreatyEngine.js:46–61`

Fixed

> commission = prem × (fixed_commission_pct / 100)

The treaty-type-aware `buildTreatyTerms` picks `fixed_commission_qs_pct` for quota-share treaties and `fixed_commission_surplus_pct` for surplus treaties (with the other as fallback).

Sliding scale — explicit table

```
LR = (cappedClaims / prem) × 100

// Linear interpolation between table rows sorted by LR ascending
if LR ≤ table[0].lr      → commission_pct = table[0].cm
if LR ≥ table[n].lr      → commission_pct = table[n].cm
else                     → linear interpolate between bracketing rows

commission = prem × (commission_pct / 100)
```

Sliding scale — min/max only

Higher loss ratio → lower commission (inverse relationship).

```
if LR ≤ minLR → commission_pct = maxC
if LR ≥ maxLR → commission_pct = minC
else          → commission_pct = maxC − ((LR − minLR) / (maxLR − minLR)) × (maxC − minC)

commission = prem × (commission_pct / 100)
```

2.4 Sliding-table interpolation helper

`client/src/logic/propTreatyEngine.js:27–44`

Generic linear-interpolation lookup; less than 2 table points returns `null`. Non-numeric points are filtered before sorting by LR ascending.

2.5 Profit commission with loss carry-forward (stateful)

`client/src/logic/propTreatyEngine.js:65–93`

Returns a closure. Each call processes one contract year in chronological order and updates a deficit queue maintained inside the closure. Callers must keep the same closure instance across all years for the LCF semantics to be correct.

Without LCF

```
mgmt    = prem × (mgmt_expenses_pct / 100)
yearPL  = prem − cappedClaims − comm − mgmt

pc = (yearPL > 0) ? yearPL × (profit_commission_pct / 100) : 0
```

With LCF (and optional extinction)

```
1.  mgmt   = prem × (mgmt_expenses_pct / 100)
2.  yearPL = prem − cappedClaims − comm − mgmt

3.  If extinction is on, evict deficits older than lcf_years.

4.  If yearPL ≤ 0:
        push { year, amount: −yearPL } to deficit queue
        return pc = 0

5.  Else burn deficits FIFO from the front of the queue:
        remaining = yearPL
        while deficits.length > 0 and remaining > 0:
          if deficits[0].amount ≤ remaining:
            remaining −= deficits[0].amount; shift()
          else:
            deficits[0].amount −= remaining; remaining = 0

6.  pc = remaining × (profit_commission_pct / 100)
```

Edge cases. `prem ≤ 0` returns 0 for the year (the year is skipped — does *not* contribute to the deficit queue). LCF years of 0 makes the queue trivially empty.

2.6 Loss participation credit (single or multi-corridor)

`client/src/logic/propTreatyEngine.js:95–124`

Cedant receives back a share of profits above a loss-ratio floor. Multi-corridor handles stepped sharing bands.

Single corridor

```
LR    = claims / prem
minLR = min_loss_ratio_pct / 100
maxLR = max_loss_ratio_pct / 100  (default 1.0)
share = reinsurer_share_pct / 100

if LR ≤ minLR → credit = 0
if LR ≥ maxLR → effLR  = maxLR
else          → effLR  = LR

credit = (effLR − minLR) × prem × share
```

Multi-corridor (lp_slides)

```
LR = claims / prem
credit = 0

for each slide in lp_slides:
  loFrac = slide.min_lr / 100
  hiFrac = slide.max_lr / 100
  share  = slide.share  / 100
  top    = min(LR, hiFrac)
  band   = max(0, top − loFrac)
  credit += band × prem × share
```

Edge cases. `lp_enabled = false` or `prem ≤ 0` returns 0. Slides with `share = 0` are filtered before the loop.

2.7 buildTreatyTerms — input normaliser

`client/src/logic/propTreatyEngine.js:129–168`

Flattens a contract payload (with nested `commissions`, `detail`, `lossParticipation`, `header`) plus an in-memory treaty detail object into the single `t` shape that the helpers above consume. The interesting bit is commission selection: it inspects the treaty type name to decide whether `fixed_commission_qs_pct` or `fixed_commission_surplus_pct` has priority, falling back to whichever is non-null. Everything else is a straight precedence merge.

3. Non-proportional (XoL) layer pricing

Per-layer pricing lives in `client/src/utils/npPricingEngine.js`. Each method returns a rate (decimal); the screen UI converts to ROL or premium for display. The three methods feed the three-way blend in §1.7 to produce a single layer total.

3.1 Pure burning cost — historical experience method

`client/src/utils/npPricingEngine.js:177–256`

The straightforward actuarial method. Cuts selected historical losses to the layer, sums by year, normalises by EGNPI, averages across the observation window, applies to prospective EGNPI.

```
1. selected = losses where is_selected ≠ false

2. For each year:
       byYear[year] = Σ layerHit(inflated_incurred ?? (incurred + os), D, L)

3. If a per-year EGNPI map is supplied (Clark alignment):
       for each year with EGNPI > 0:
         lossCostPct[year] = byYear[year] / egnpiByYear[year]
         totalLossCost += lossCostPct[year]
       avgLossCost = totalLossCost / obsYears
   Else (single prospective EGNPI):
       avgLossCost = (Σ byYear values / obsYears) / egnpi

4. avgAnnualLayerLoss = avgLossCost × prospectiveEgnpi
5. rol = avgAnnualLayerLoss / limit  ≡ avgLossCost
       (rate on EGNPI ≡ ROL on limit when normalised by limit)
```

Edge cases. No selected losses, `limit ≤ 0`, or `obsYears ≤ 0` all return ROL = 0. The observation window **must** include zero-loss years — see the calibration trap in §1.3.

3.2 Pareto pricing — heavy-tail method

`client/src/utils/npPricingEngine.js:274–311`

Fits a Pareto distribution to selected losses (or reuses pre-fitted params from the loss-selection screen), then prices the layer using the Limited Expected Value formula. Preferred for cat tail risk where large losses dominate.

Threshold (xm) and MLE fit

```
xm = 25th percentile of selected loss values

For losses x_i ≥ xm:
  α = n / Σ_i log(x_i / xm)        (MLE for shape parameter)
```

If `savedParams.pareto_alpha` and `pareto_xm` are both positive, the pre-fitted snapshot from the loss-selection screen is used instead of re-fitting.

Layer LEV and expected loss

```
E[min(X, c)] for Pareto(α, xm):
  c ≤ xm      → c
  α ≈ 1       → xm × (1 + ln(c/xm))
  otherwise   → (xm·α / (α−1)) · (1 − (xm/c)^(α−1)) + c · (xm/c)^α

E[layer[D, L]]   = E[min(X, D+L)] − E[min(X, D)]
D_eff            = max(D, xm)     // Pareto only models X ≥ xm
expLayerLoss     = (n / obsYears) × ( LEV(D_eff + L) − LEV(D_eff) )
rol              = expLayerLoss / egnpi
```

Attachment / exhaustion probabilities

```
prAttach  = (xm / D)^α        if D ≥ xm, else 1
prExhaust = (xm / (D+L))^α
```

Edge cases. Fewer than 3 selected losses, `limit ≤ 0`, `egnpi ≤ 0`, or a non-positive fitted α all return ROL = 0.

3.3 Risk exposure rating — Swiss Re MBBEFD

`client/src/utils/npPricingEngine.js:355–400`

Exposure rating for risk XL. Drives expected layer loss from policy bands using the Swiss Re Y-curve family. Each band's average sum-insured selects the curve parameter `c` (or the user picks one explicitly).

Swiss Re curve thresholds

Curve	Segment	Avg SI threshold	c
Y1	Personal lines	≤ 400k	0
Y2	Commercial small	≤ 1M	1.5
Y3	Commercial medium	≤ 2M	3.0
Y4	Industrial / large	> 2M	5.0

G function (damage-ratio CDF)

```
G(d, c) where d ∈ [0, 1]:
  d ≤ 0    → 0
  d ≥ 1    → 1
  |c| < 1e-10 → d                              (linear at c ≈ 0)
  otherwise   → log(1 + (e^c − 1) · d) / c
```

Algorithm

```
1. For each band: avgSI = total_sum_insured / no_of_risks
2. Pick curve (Auto/Y1..Y4/Custom) → c
3. Band LEV using PML as cap:
      LEV = SI · (PML% / 100) · [ G((D+L)/PML, c) − G(D/PML, c) ]
4. Band expected loss = LEV × no_of_risks
5. totalExpLoss = Σ band expected losses
6. Apply gross loss ratio (step 8 of the Swiss Re brochure):
      totalExpLoss ×= gross_loss_ratio / 100
7. rol = totalExpLoss / egnpi
```

Edge cases. No bands → 0. `limit ≤ 0` or `egnpi ≤ 0` → 0. A zero or missing gross loss ratio defaults to 100% (no adjustment), which is the safe direction.

3.4 Cat exposure rating — three methods, priority-ordered

`client/src/utils/npPricingEngine.js:423–505`

Tries each method in order; the first one with usable data wins. The "no usable data" exit returns `method = 'none'` so the UI can show "un-priced".

Method 1 — return-period (OEP) curve (preferred)

Trapezoidal integration of the layer hit against the OEP curve (Annual Expected Layer Loss).

```
points = key OEP points from cat snapshot (RP10, RP25, RP50, RP100, RP200)
         padded/synthesised where missing (e.g. RP1=0, RP5=0.4·RP10)

aep = 0
for i in 0 .. points.length − 2:
  p1, p2 = points[i], points[i+1]
  f1     = 1 / p1.rp                  // exceedance frequency
  f2     = 1 / p2.rp
  h1     = layerHit(p1.loss, D, L)
  h2     = layerHit(p2.loss, D, L)
  aep   += (f1 − f2) · (h1 + h2) / 2  // trapezoidal rule on the OEP

rol = aep / egnpi
```

Method 2 — Pareto fit (fallback)

Identical to §3.2 but consumes the catastrophe loss array rather than the risk loss array. Requires at least three selected cat losses.

Method 3 — flat 15% loss ratio (last resort, gated on CRESTA presence)

Only fires when CRESTA aggregates (`eq_agg + ws_agg + flood_agg + srcc_agg + others_agg`) are non-zero AND neither method 1 nor method 2 is available. This gate is important: it prevents the engine from silently quoting *any* cat layer even when there's no exposure data at all.

```
totalExposure = Σ CRESTA aggregates
if totalExposure > 0:
  impliedLoss = egnpi × 0.15
  rol = max(0, min(impliedLoss − D, L)) / (egnpi × obsYears)
else:
  rol = 0   method = 'none'
```

Attachment / exhaustion (all methods)

Interpolated linearly from the OEP curve points when method 1 ran; otherwise the Pareto closed-form is used (method 2) or both are zero (method 3).

3.5 Large-loss development (separation)

Loss triangles are split into attritional (below a threshold) and large (at or above) cohorts so each can follow its own age-to-age pattern and tail factor. The mechanics are the chain ladder of §4 applied twice; the only domain-specific decision is the threshold, which the loss-selection screen exposes.

4. Development factors & triangles

Chain-ladder utilities live in `client/src/logic/chainLadder.js`. The library is used by every loss/premium triangle viewer and underlies the IBNR projections on the dev factor screens.

4.1 Age-to-age factors per cell

`client/src/logic/chainLadder.js:32–41`

```
factor[row][col] = matrix[row][col+1] / matrix[row][col]      if both non-null
                                                                  and matrix[row]
[col] ≠ 0
                 = null                                            otherwise
```

4.2 Pattern — one factor per development column

`client/src/logic/chainLadder.js:58–93`

Three averaging methods reduce the per-cell factors of §4.1 to one factor per column:

Method	Definition
`weighted`	`LDF[c] = Σ matrix[*][c+1] / Σ matrix[*][c]` (column totals)
`last3`	Arithmetic mean of the last three rows' factors at column `c`
`last5`	Arithmetic mean of the last five rows' factors at column `c`

The function flags columns with fewer than three contributing rows as volatile — the UI surfaces these as warnings rather than failing the projection.

4.3 Cumulative development factors (CDFs)

`client/src/logic/chainLadder.js:106–111`

```
CDF[n] = tailFactor                        // last column → ultimate
for i = n−1 down to 0:
  CDF[i] = pattern[i] × CDF[i+1]
```

`CDF[0]` represents all remaining development; `CDF[n]` is just the tail factor (development beyond the triangle).

4.4 Project to ultimate

`client/src/logic/chainLadder.js:123–138`

```
cdfs = calculateCdfs(pattern, tailFactor)

For each origin year:
  latestCol = last non-null column on that row
  if latestCol == −1   → ibnr = 0  (no data)
  ultimate  = matrix[row][latestCol] × cdfs[latestCol]
  ibnr      = ultimate − matrix[row][latestCol]
```

4.5 Exponential tail fit

`client/src/logic/chainLadder.js:140–159`

Fits the model `CDF[i] = 1 + exp(a + b · (i+1))` by linear regression on the log-shifted CDFs. Extrapolates one or more "phantom" columns past the observed data so that the tail factor isn't a single arbitrary number pulled from thin air.

1. `y_i = log(CDF[i] − 1)        for CDF[i] > 1.000001`
2. `Fit y = a + b·(i+1) by ordinary least squares`
3. `Extrapolate CDF[i] = 1 + exp(a + b·(i+1))`

Edge cases. Fewer than two CDFs above 1.000001 returns the original CDFs (no fit attempted). A degenerate regression denominator (≈ 0) does the same. The final CDF is forced to `1.0` if the original was already `1.0` (ultimate reached — don't fabricate further development).

4.6 Derive LDFs from CDFs (inverse of 4.3)

`client/src/logic/chainLadder.js:161–168`

```
LDF[i] = CDF[i] / CDF[i+1]    if both finite and CDF[i+1] ≠ 0
       = null                  otherwise
```

Lets the triangle viewers toggle between LDF and CDF display modes without losing precision.

5. Effective premium & exposure (FX-aware)

Dashboard and home aggregates do not store premium amounts in any single currency; they derive them at query time from the contract's currency, the latest stored exchange rate, and an optional display currency override. The maths is the same for premium and limit — only the source field changes.

5.1 effPremUSD (or any target display currency)

`server/src/routes/dashboard.js:90–100 · server/src/routes/home.js:153–164`

```
linePct  = COALESCE(
             c.signed_line_pct,
             (latest contract_offer.written_line_pct),
             100.0
           ) / 100

basePrem = CASE
             WHEN treaty is NP  → nd.est_gnpi
             ELSE               → pd.quota_share_epi + pd.surplus_epi
           END

fxRate     = COALESCE(fx.rate_to_usd, 1.0)             // contract ccy → USD
displayDiv = targetRateToUsd                            // USD → display ccy

effPremUSD = basePrem × linePct × fxRate / displayDiv
```

5.2 effLimUSD

`server/src/routes/dashboard.js:102–113`

Identical structure with limit instead of premium:

```
baseLimit = CASE
              WHEN treaty is NP  → SUM(contract_np_layers.layer_limit)
              ELSE               → pd.total_capacity
            END

effLimUSD = baseLimit × linePct × fxRate / displayDiv
```

5.3 Signed-line fallback chain

1. `c.signed_line_pct` — set when the contract is countersigned
2. The most recent `contract_offer.written_line_pct` — order by `updated_at DESC LIMIT 1`
3. Default `100%` — treat as fully written if no signal exists

5.4 Country aggregate weighted by signed line

server/src/modules/pricing/repositories/pricingAggregateRepository.js:51

```
weighted_agg = Σ ( cresta.total_agg × COALESCE(signed_line_pct, written_line_pct, 0)
/ 100 )
```

Used for CRESTA overlap detection across the portfolio.

6. Dashboard & home aggregates

6.1 Status KPIs (home page)

`server/src/routes/home.js:209–220`

KPI	SQL
`waiting_approval`	`COUNT(* WHERE uw_status = 'WAITING_APPROVAL')`
`waiting_signed_line`	`COUNT(uw_status IN ('AWAITING_SIGNED_LINE', 'APPROVED'))`
`signed`	`COUNT(uw_status = 'SIGNED')`
`declined` / `ntu` / `drafts`	One status each
`total`	Sum across all status buckets

6.2 Pivot aggregation

`server/src/routes/dashboard.js:274–295`

Generic cross-tab builder. Rows are the bigger dimension (region or LOB), columns the smaller (treaty type or year). The function returns `{ columns, rows, totals }` where each row has its own per-column values plus a row total, and there's a grand-total column-vector across the bottom.

6.3 Region bucket mapping

`server/src/routes/dashboard.js:61–68` · `server/src/routes/home.js:143–152`

`country.region`	Bucket
GCC, Levant, North Africa	Middle East
Sub-Saharan Africa	Africa
South Asia, Southeast Asia, East Asia & Pacific	Asia
Europe	Europe
Americas	Americas
(anything else)	Other (excluded from portfolio views)

7. Pricing verification & drift checking

On every save of NP final pricing the server re-runs the math against the submitted output and looks for divergence. By default this is observe-only; opt-in strict mode rejects the save.

7.1 Tolerances

`server/src/lib/pricingVerifier.js:53–59`

Check	Threshold
Absolute	0.0002 (2 basis points)
Relative	0.002 (0.2% of expected)
Weight sum (burn + pareto + exposure)	100 ± 0.5 percentage points

```
diff = |stored − expected|
PASS if diff ≤ 0.0002
   or |expected| > 0 and diff / |expected| ≤ 0.002
```

7.2 Modes

- **Default (warn-only)** — every response carries `X-Pricing-Drift-Count`, and structured logs record `{ endpoint, pricingDriftCount, maxAbsDiff, driftMagnitudeBucket, contractId, quoteId }`. The save still completes.
- **Strict (`PRICING_STRICT=1`)** — the request is rejected with `422 PRICING_DRIFT` and the response body lists the drifting fields.

7.3 Weight-sum check

`server/src/lib/pricingVerifier.js:100–111`

```
weightSum = burn_weight_pct + pareto_weight_pct + exposure_weight_pct
DRIFT if |weightSum − 100| > 0.5
```

8. FX / currency conversion

8.1 Storage

`ref_exchange_rate` is the single source of truth. Columns: `currency_code`, `rate_to_usd` (decimal), `effective_date`, `source` (MANUAL/SYSTEM).

8.2 Lookup — latest rate per currency

```
SELECT rate_to_usd
FROM   public.ref_exchange_rate
WHERE  currency_code = $1
ORDER  BY effective_date DESC
LIMIT  1
```

8.3 Multi-leg conversion (contract → USD → display)

```
// Contract SAR → display GBP
rate_to_usd_sar = lookup('SAR')      // e.g. 0.267
rate_to_usd_gbp = lookup('GBP')      // e.g. 1.27

amount_USD = amount_SAR × rate_to_usd_sar
amount_GBP = amount_USD / rate_to_usd_gbp

// Combined
amount_GBP = amount_SAR × rate_to_usd_sar / rate_to_usd_gbp
```

This is exactly the path baked into the `effPremUSD` and `effLimUSD` SQL of §5: the contract-side `fx.rate_to_usd` multiplies, the display-side `targetRateToUsd` divides.

9. Proportional pricing constants & helpers

9.1 Swiss Re G function (shared with §3.3)

`client/src/screens/proportional/pricing/components/propPricingConstants.js:58–62` · `client/src/utils/npPricingEngine.js:58–65`

Defined twice in the codebase (one place for prop-side, one for NP-side); the math is identical. See §3.3 for the formula.

9.2 Pareto helpers

`client/src/screens/proportional/pricing/components/propPricingConstants.js:66–75`

Used on the loss-selection screen to fit a tail and estimate return-period quantiles.

```
fitPareto(losses, xm) → { alpha, n }
   alpha = n / Σ log(loss_i / xm)             for loss_i ≥ xm

paretoQ(p, alpha, xm) → quantile at exceedance probability p
   Q(p) = xm · (1 − p)^(−1/α)
```

10. Workbench formula governance

The workbench (`server/src/routes/workbench.js`) is a parameter registry, not a formula evaluator. It tracks values used by the formulas in the other sections (Swiss Re curve multipliers, target loss ratios, loadings, etc.) and applies a submit/approve workflow on top.

10.1 Tables

- `formula_parameters` — `{module, formula_name, parameter_key, current_value, pending_value, status}`
- `formula_change_log` — `{formula_parameter_id, action, old_value, new_value, changed_by, changed_at, approved_by, approved_at}`
- `formula_comments` — free-text notes per parameter.

10.2 Workflow

1. **Submit** — TD+ (`hierarchy_level ≤ 3`) proposes a new value; `pending_value` is set, status becomes `PENDING`.
2. **Approve** — CU/CE (supervisor) promotes `pending_value → current_value`, status `APPROVED`.
3. **Reject** — supervisor leaves `current_value` in place, status `REJECTED`.
4. **History** — every transition is logged with timestamps and actors.

No user-defined expression evaluation is wired up; the workbench is governance for numeric constants only.

11. Client-side number parsing

11.1 parseFlexibleNumber

`client/src/utils/format.js:195–217`

Every numeric form input runs values through this parser. It accepts pretty much anything a user is likely to type or paste from a spreadsheet.

1. Strip currency symbols and locale spaces.
2. Detect the decimal separator (last dot or comma, with magnitude > 1000).
3. `parseFloat`.
4. Reapply negation if the original was parenthesised (accounting negative).

Examples that all parse to `-500`: `(500)`, `-500`, `-$500.00`, `-500,00 EUR`.

Empty, null, and NaN return `null`. The convenience wrapper `toN(v) = parseFlexibleNumber(v) ?? 0` coerces to zero where pricing math expects a number.

12. Pricing component snapshots

`server/src/routes/pricing.js:82–84 · pricingRepository.js`

Persists a named snapshot of NP final-pricing inputs and outputs at a point in time so underwriters can compare iterations without re-running the engine. The fields mirror the inputs of §1.7 plus the resulting `total_price`:

- `layer_number`, `section` (`RISK` | `CAT`)
- `pure_burning_cost`, `pareto_pricing`, `exposure_rating`
- `burn_weight_pct`, `pareto_weight_pct`, `exposure_weight_pct`
- `pricing_loading_pct`
- `total_price` (computed at save time via §1.7)
- `snapshot_name`, `created_at`, `created_by`

The verifier of §7 is what guarantees `total_price` remains consistent with the stored components — if a future migration or client change ever produced a different blended total, the drift logs would catch it.

13. Critical notes & caveats

1. **Zero-loss years are mandatory in burning cost.** `annualiseLoss` divides by the total calendar window, not active years. The most common calibration error is computing "average loss across the four years that actually had a claim" instead of the ten-year observation window.
2. **Three-way blend vs. legacy two-way.** The codebase still contains an older two-way blend in places (burn + exposure only). The current authoritative formula for NP final pricing is the three-way `deriveComponentTotal` of §1.7.
3. **Loading ≥ 100% throws.** Recently hardened. Previously `applyLoading` silently returned 0 for absurd loadings; it now raises `RangeError` so the bad input surfaces upstream rather than being baked into a snapshot.
4. **Pareto threshold xm is derived, not user-input.** Fitted at the 25th percentile of selected losses unless the loss-selection screen has already saved `pareto_alpha` + `pareto_xm`, in which case those win.
5. **FX is a multi-leg path.** Always `contract_ccy → USD → display_ccy`. The denominator in `effPremUSD / effLimUSD` is the display currency's rate, not the contract currency's.
6. **Signed-line priority.** Signed line beats the latest written line, which beats a default of 100%. Don't read raw EPI off the contract — it's only meaningful after this fallback chain.
7. **Treaty type is detected via SQL `ILIKE`.** Dashboard SQL keys off `tt.category ILIKE '%NP%' OR ILIKE '%NON%'`. New treaty categories that don't match either substring will be misclassified as proportional — be careful when seeding new categories.
8. **Cat exposure rating fallback is gated.** Method 3 only fires when CRESTA aggregates exist. If they don't, `method='none'` propagates to the UI so operators see "un-priced" rather than a phantom 15% number.
9. **ROL vs. rate.** ROL is dimensionless (premium / limit). The "rate" the engine returns from burning cost is also a fraction of EGNPI — equal to ROL after `annualLoss / limit` normalisation. The terms are used interchangeably in the codebase; always check the denominator.
10. **Pricing verifier is the safety net for the shared library.** Because `shared/pricingMath.js` is imported by both client and server, `pricingVerifier.js` can re-derive the stored value and compare bit-by-bit (within tolerance). Any divergence either means the client lied or the math file changed without a re-save — both worth investigating.

14. Summary table — every formula in one view

Function	Where	Inputs	Returns	Key edge case
`layerHit`	shared/pricingMath.js	loss, D, L	currency	L ≤ 0 → 0
`weightedAverage`	shared/pricingMath.js	values[], weights[]	number or null	total weight ≤ 0 → null
`annualiseLoss`	shared/pricingMath.js	totalLoss, years	currency / yr	years ≤ 0 → 0
`rolFromAnnualLoss`	shared/pricingMath.js	annualLoss, limit	decimal (0..1)	limit ≤ 0 → 0
`premiumFromRol`	shared/pricingMath.js	rol, limit	currency	—
`applyLoading`	shared/pricingMath.js	rate, loading%	decimal	loading ≥ 100 → throw
`deriveComponentTotal`	shared/pricingMath.js	burn, pareto, exp, 3×weight, loading	decimal	Σ weights = 0 → 0
`applyLossCap`	propTreatyEngine.js	claims, prem, t	currency	prem ≤ 0 → claims
`calcComm`	propTreatyEngine.js	prem, claims, t	currency	prem ≤ 0 → 0
`makePCCalc`	propTreatyEngine.js	stateful per-year	closure → currency	chronological order required
`calcLPC`	propTreatyEngine.js	prem, claims, t	currency	!enabled → 0
`calcPureBurningCost`	npPricingEngine.js	losses, D, L, egnpi, obsYears	{ rol, … }	no selected → rol 0
`calcParetoROL`	npPricingEngine.js	losses, D, L, egnpi	{ rol, α, xm }	< 3 losses → 0
`calcRiskExposureRating`	npPricingEngine.js	profiles, D, L, egnpi	{ rol, … }	no bands → 0
`calcCatExposureRating`	npPricingEngine.js	cresta, D, L, egnpi, snap, losses	{ rol, method }	no data → method 'none'
`calculateAgeToAgeFactors`	chainLadder.js	matrix	factors[][]	zero divisor → null cell
`calculatePattern`	chainLadder.js	matrix, factors, method	{ pattern, warnings }	< 3 rows → warning
`calculateCdfs`	chainLadder.js	pattern, tailFactor	cdfs[]	—
`projectToUltimate`	chainLadder.js	pattern, tail, data	{ cdfs, projections }	empty row → ibnr 0
`effPremUSD`	dashboard.js / home.js	linePct, basePrem, fxRate, displayDiv	currency	missing fx → 1.0
`effLimUSD`	dashboard.js	linePct, baseLimit, fxRate, displayDiv	currency	same
`buildPivot`	dashboard.js	rows, rowKey, colKey, valKey	{ columns, rows, totals }	excludes 'Other' / null
`weighted_country_agg`	pricingAggregateRepository.js	cresta_agg, signed_line_pct	currency	—